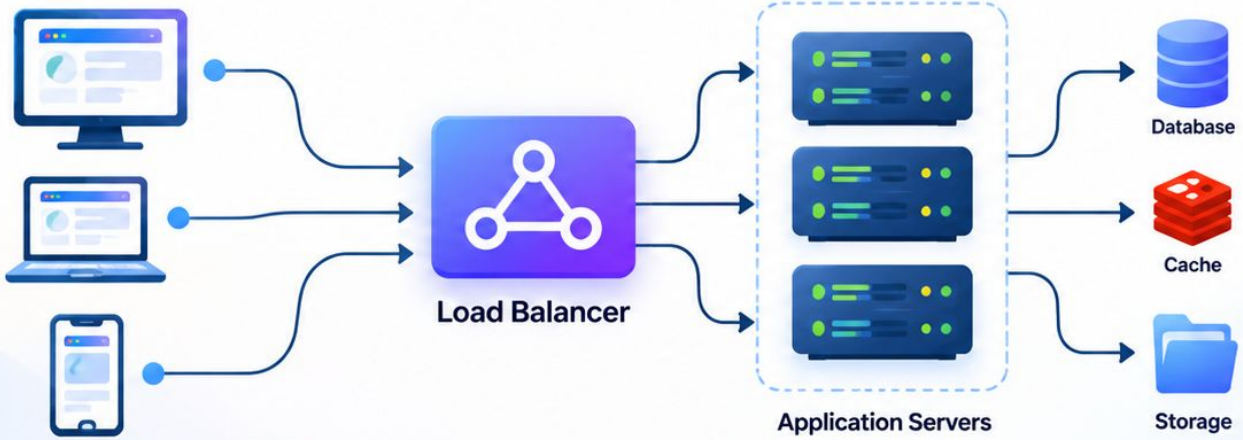
Introduction to Scalability

Scalability in System Design

Mastering System Design: From Basics to Cracking Interviews





Instructor: Rahul Rajat Singh

What is Scalability?

Scalability is the ability of a system to handle an increasing amount of work, or its potential to accommodate growth.

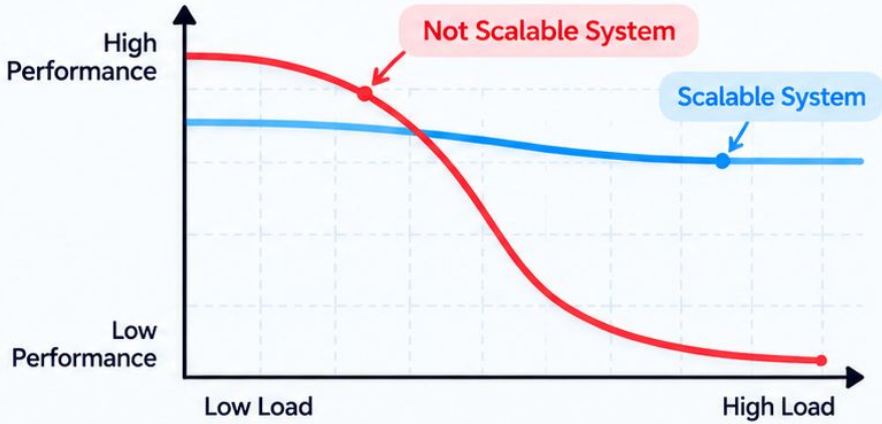
It ensures **performance, reliability**, and **availability** under growing load.



Why Do Systems Need to Scale?

1. **User base growth**
   e.g., launching in new regions
2. **Increasing data volume**
   e.g., IoT, analytics
3. **Peak events**
   e.g., Black Friday, ticket sales
4. **Avoid service degradation**
   or downtime
5. **Meet performance SLAs**

Performance vs. Load





Types of Scalability (Intro)

1 Vertical Scaling

Add more CPU/RAM to one server.



Small Server
2 CPU | 4 GB RAM

Bigger Server
8 CPU | 32 GB RAM

2 Horizontal Scaling

Add more servers to distribute the load.



We'll go deep into these types in the next lecture.

Common Challenges in Scaling

Latency

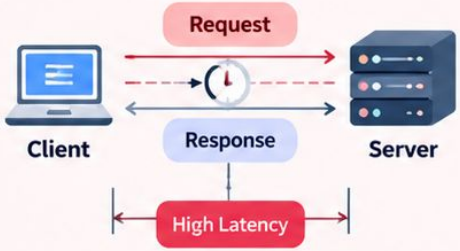
Delay between request and response.

Common Causes

- Network hops
- Slow database queries
- Synchronous calls

Common In

- Amplified in microservices and distributed systems



Bottlenecks

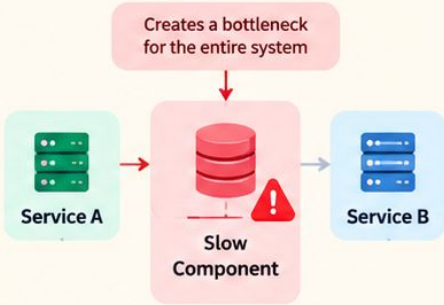
One slow component can cause system-wide slowdown.

Examples

- Database locks
- Memory limits
- Single-threaded processing

Challenge

- Hard to predict as load grows



Downtime

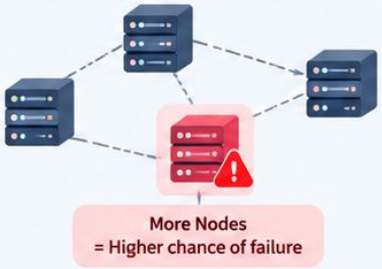
More nodes and components mean more failure points.

Common Causes

- More nodes = more failure points
- Updates, redeployments,
- scaling events can cause outages

Impact

- High availability becomes harder



Cost

Infrastructure isn't free — scaling has real costs.

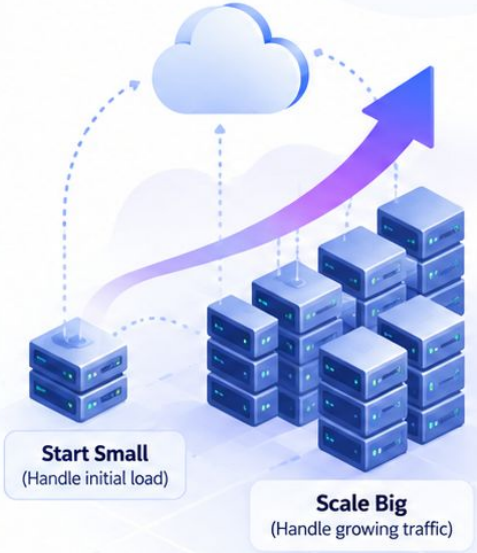
Cost Factors

- CPU, RAM, bandwidth, etc.
- Autoscaling without limits = budget nightmare
- Over-provisioning = wasted spend



Interview Questions - Scaling

1. What does **scalability** mean in the context of system design?
2. Can you explain a **real-world example** where scalability was critical to success or failure?
3. What are the main **challenges** systems face as they scale?
4. How would you identify a **bottleneck** in a scalable architecture?
5. Why does **latency** increase with scale, and how can you mitigate it?
6. How do you balance scalability with **cost** in cloud-based systems?



Think beyond adding servers: identify bottlenecks, latency, reliability, and cost.

Instructor: Rahul Rajat Singh

Summary & What's Next

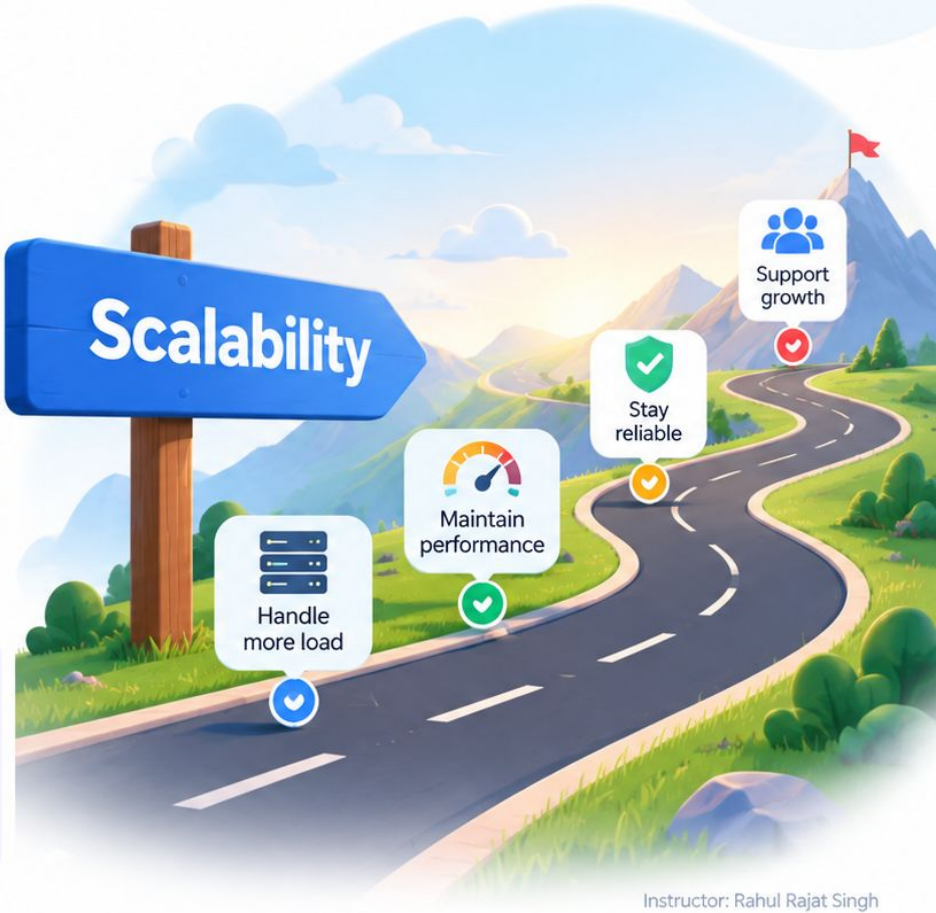
1. **Scalability** helps systems **grow** without breaking.

2. **Two main types**: **vertical** and **horizontal**.

3. But it introduces **latency**, **cost**, and **other risks**.

What's Next?

Scaling Strategies — how to actually implement horizontal, vertical, and diagonal scaling with trade-offs and real-world choices.



Scaling Strategies: Horizontal, Vertical & Diagonal

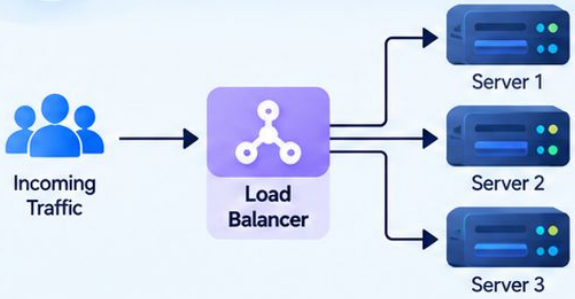
Scalability in System Design

Mastering System Design: From Basics to Cracking Interviews

Rahul Rajat Singh

Horizontal Scaling

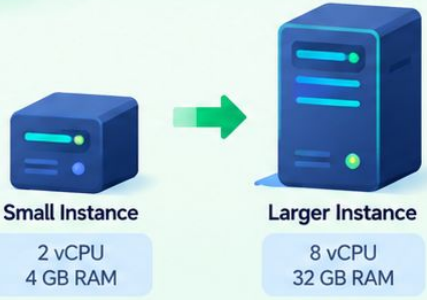
Add more machines (instances)



- Handles increased traffic by adding more servers.
- Common in web applications and microservices (behind a load balancer).

Vertical Scaling

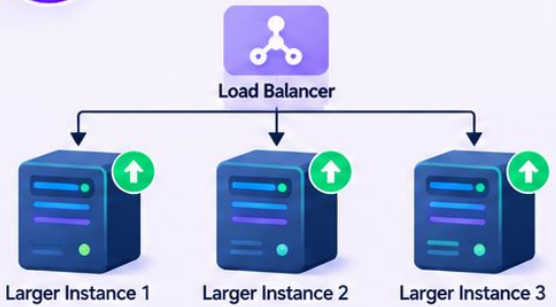
Add more power to a single machine



- Increase CPU, RAM, storage on the same machine.
- Simple to implement, but has hardware limits.

Diagonal Scaling

Combine vertical + horizontal scaling



- Add more powerful machines and more machines.
- Best for large-scale, high-traffic systems.

Types of Scalability – Deep Dive

1 Vertical Scaling (Scale Up)



- Upgrade one machine (CPU, RAM, Disk)
- Easy to implement (less complexity)
- Limits: Physical cap, risk of single point of failure

2 Horizontal Scaling (Scale Out)



- Add more machines/nodes
- Distribute traffic/load across servers
- Requires load balancer, stateless design
- Complex setup (coordination, replication)

3 Diagonal Scaling (Hybrid)



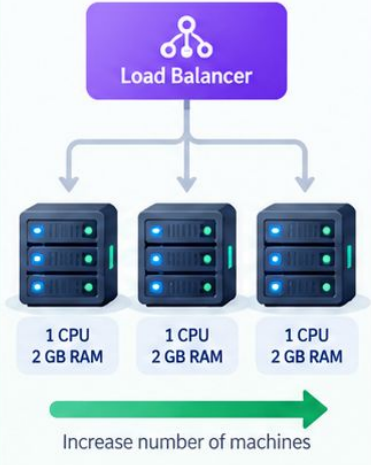
- Start vertical, then go horizontal
- Hybrid approach: start vertical, add horizontal as needed
- Common in cloud-native apps
- Cost-effective + long-term ready

VERTICAL SCALING
Scale Up



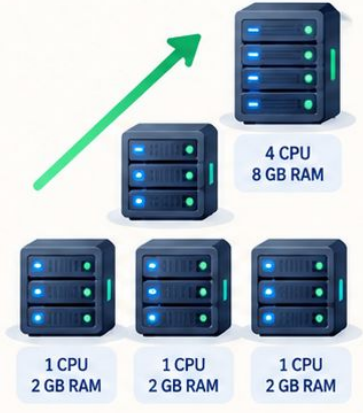
- More power in one server
- Simple to implement
- Limited by hardware
- Single point of failure risk

HORIZONTAL SCALING
Scale Out



- Add more machines
- Distribute traffic/load
- Requires load balancer
- Stateless design
- Complex setup (coordination)

DIAGONAL SCALING
Scale Up + Scale Out



- Start vertical, then go horizontal
- Hybrid approach: add both as needed
- Common in cloud-native apps
- Cost-effective + long-term ready

Trade-Offs: Cost vs Complexity vs Performance

1 Vertical Scaling

Increase the capacity of a single server (more CPU, RAM, storage).



Cost	Low-Mid
Complexity	Low
Performance	Medium

2 Horizontal Scaling

Add more servers to handle the load (scale out).



Cost	High
Complexity	High
Performance	High

3 Diagonal Scaling

Combine vertical and horizontal scaling (bigger servers + more servers).



Cost	Medium
Complexity	Medium
Performance	High

Key Takeaway More performance usually means more **cost** or **complexity**.

Instructor: Rahul Rajat Singh

Real-World Examples and When to Choose What?

Examples

Twitter
Moved from monolith → horizontal scaling (microservices)

Small startups
Vertical scaling for MVP

AWS Lambda apps
Start diagonal with autoscaling

When to Choose

Startups
Vertical (cheaper, simpler)

Scaling apps
Horizontal (resilience + capacity)

Cloud-native
Diagonal (flexibility + cost balance)

Vertical Scaling

Increase the power of a single server (better CPU, RAM, storage)



Horizontal Scaling

Add more servers to handle higher load (resilience + capacity)



Diagonal Scaling (Cloud-native)

Combine vertical and horizontal scaling with autoscaling



Decision Guide

- Consider your **stage** (MVP or scale)
- Think about **resilience** needs
- Evaluate **flexibility**
- Balance with **cost**

Interview Questions - Scaling Strategies

1. What is the difference between **horizontal** and **vertical** scaling?
2. What is **diagonal scaling** and when is it a good idea?
3. What are the **trade-offs** between horizontal and vertical scaling in terms of performance and complexity?
4. Can you describe a **scenario** where horizontal scaling wouldn't help?
5. When would you choose **vertical scaling** over horizontal?
6. What **challenges** arise in horizontal scaling and how would you solve them?



Summary & What's Next

Scaling Strategies

Vertical Scaling



Simpler, but hits limits

Horizontal Scaling



Scalable, but needs planning

Diagonal Scaling



Hybrid approach for cloud success

What's Next?

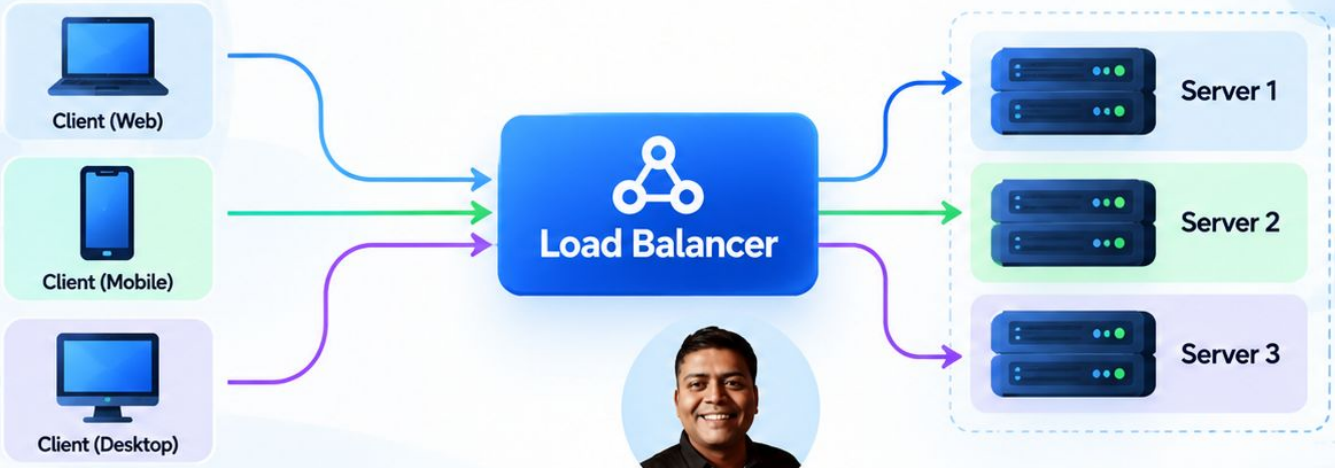
Understanding Load Balancers:
Types, Algorithms & Cloud Solutions



Understanding Load Balancers

Scalability in System Design

Mastering System Design: From Basics to Cracking Interviews





Rahul Rajat Singh

Why Load Balancing is Needed

High Availability
Ensures system uptime even under high traffic.

Traffic Distribution
Spreads requests evenly across servers.

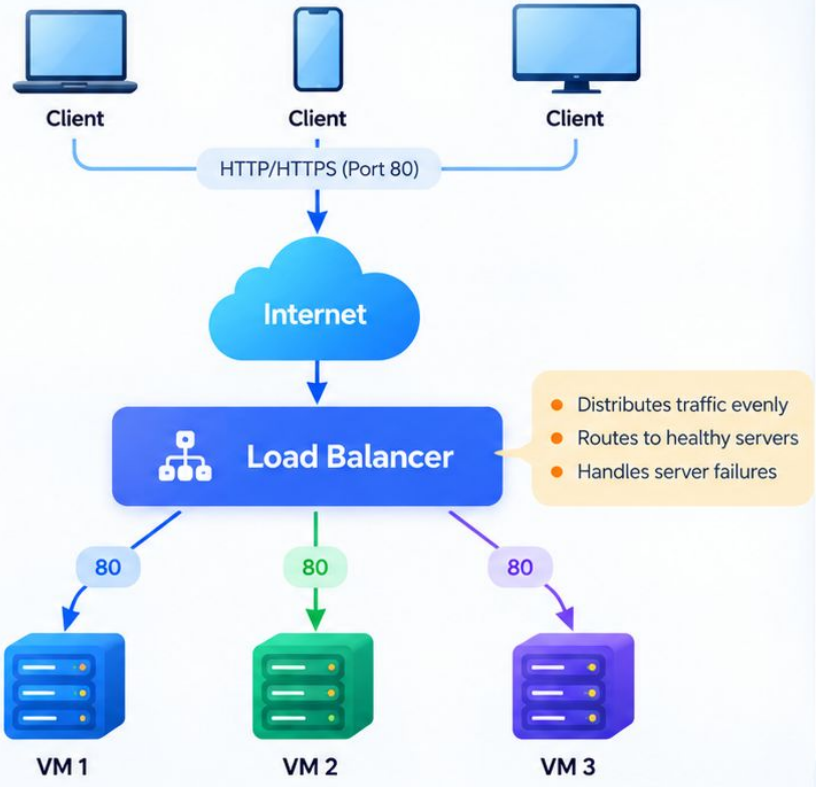
Prevents Overload
Avoids overburdening a single server.

Improves Performance
Reduces latency and enhances response times.

Handles Failures Gracefully
Redirects traffic in case of server failure.

Supports Scalability
Helps scale systems efficiently.

Real-World Example
A high-traffic e-commerce site uses load balancing to handle peak-hour requests.



Types of Load Balancers

Based on Layer

L4 TCP/UDP

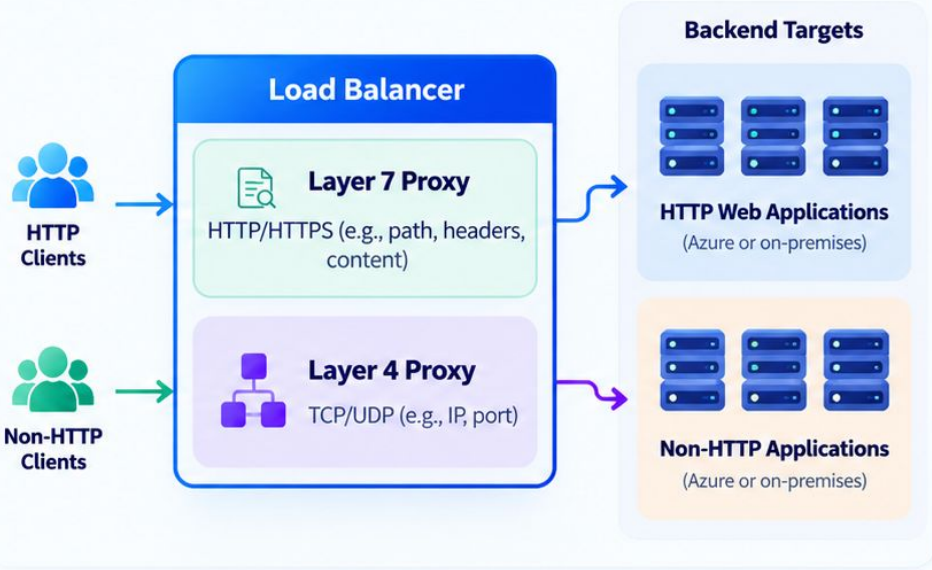
1. **Layer 4 (Transport Layer)**
   Operates at TCP/UDP level, distributing requests based on network-level data.

L7 HTTP/HTTPS

2. **Layer 7 (Application Layer)**
   Operates at HTTP/HTTPS level, making routing decisions based on request content.

Based on Deployment

1. **Hardware Load Balancers**
   Specialized devices (e.g., F5, Citrix NetScaler).
2. **Software Load Balancers**
   Nginx, HAProxy, Envoy.
3. **Cloud-based Load Balancers**
   AWS Elastic Load Balancer (ELB), Google Cloud Load Balancing.



Key Takeaway

Choose the right load balancer type and deployment model based on your application's protocol, routing needs, and scale.

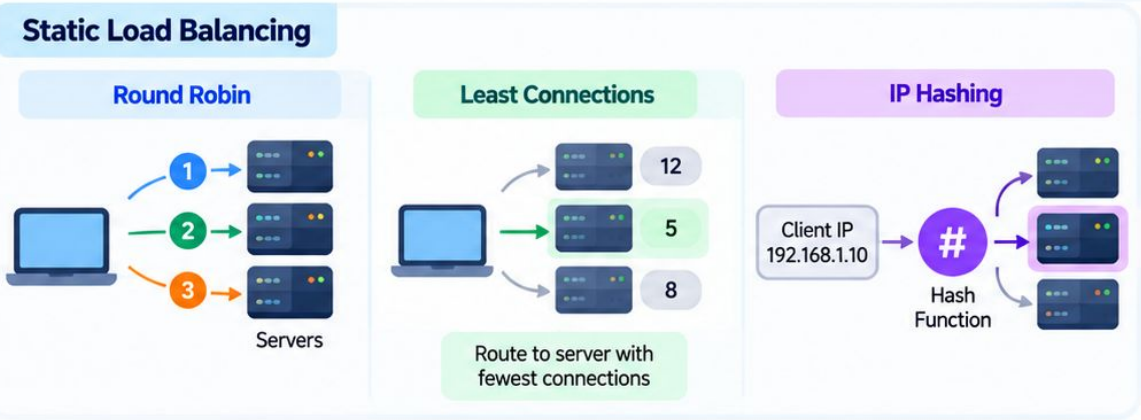
Load Balancing Strategies

1 Static Load Balancing

Round Robin
Distributes requests sequentially to each server.

Least Connections
Directs traffic to the server with the fewest connections.

IP Hashing
Routes requests based on client IP.

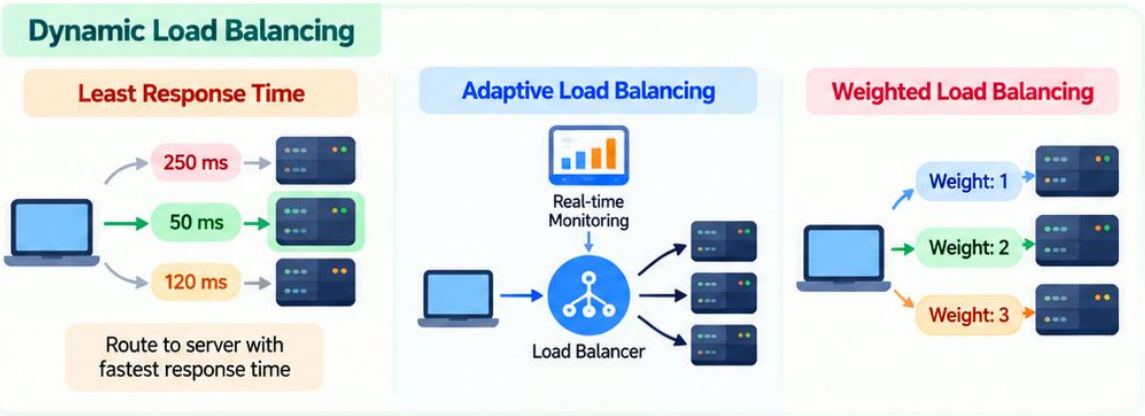


2 Dynamic Load Balancing

Least Response Time
Sends requests to the server with the fastest response.

Adaptive Load Balancing
Uses real-time monitoring to make decisions.

Weighted Load Balancing
Assigns different weights to servers based on capacity.

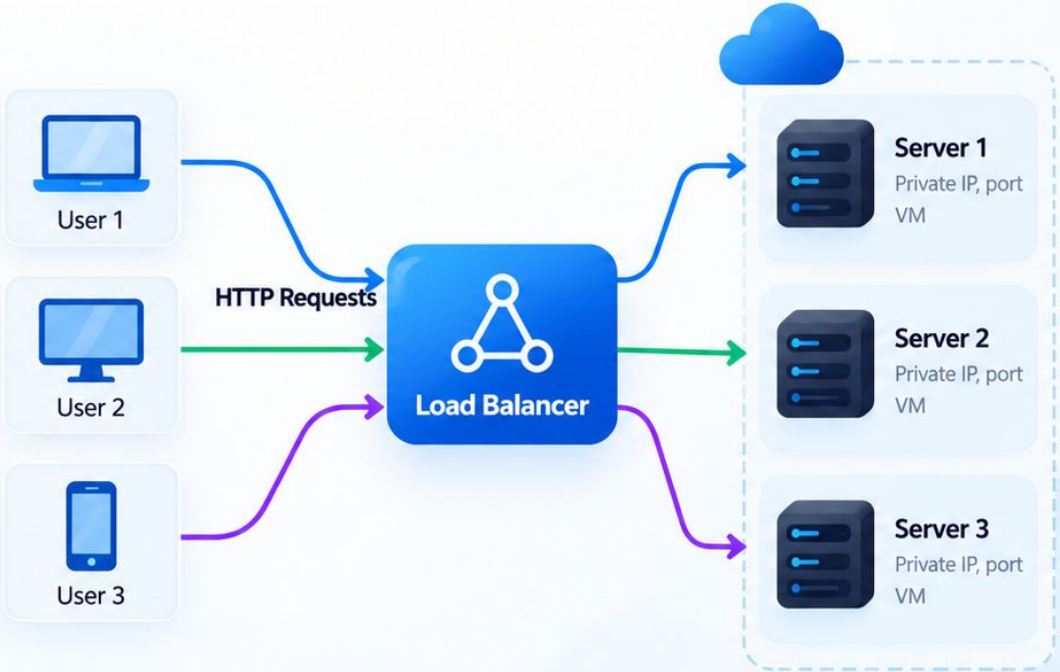


Instructor: Rahul Rajat Singh

Load Balancer in Action

Example Scenario

1. A web application with multiple servers.
2. Users send requests, and the load balancer distributes them efficiently.



Choosing the Right Load Balancer

Layer 4 vs. Layer 7

When to use each type based on your application needs.

Scalability Needs

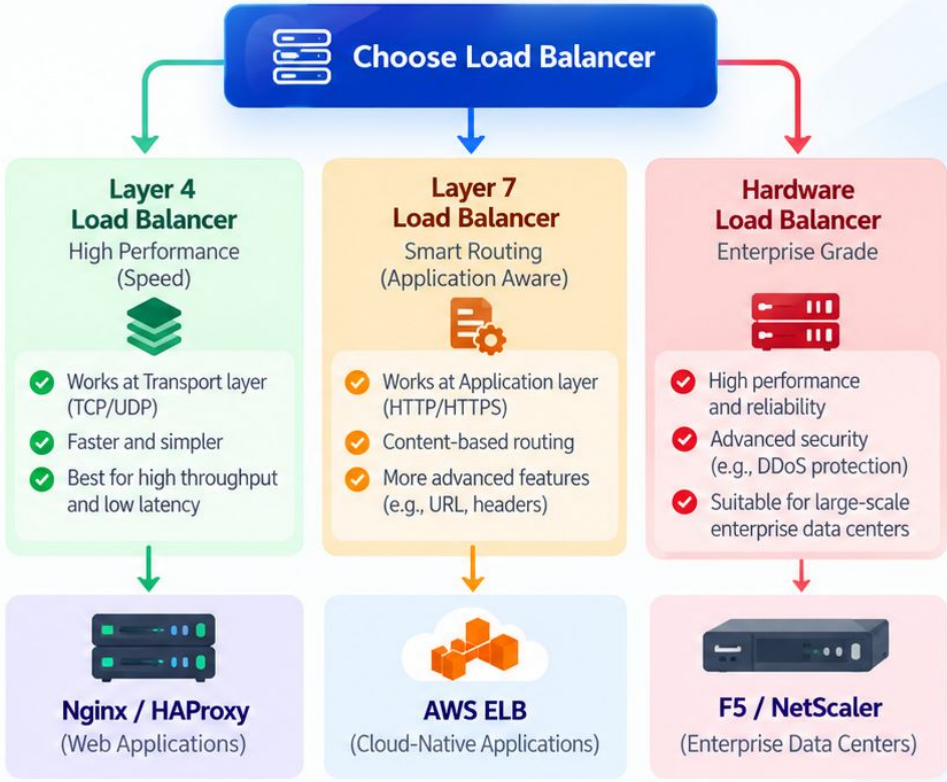
Match the right load balancer to your traffic volume and scaling requirements.

Security Concerns

Look for SSL termination and DDoS protection capabilities.

Common Use Cases

- Nginx/HAProxy for web applications.
- AWS ELB for cloud-native applications.
- Hardware Load Balancers for enterprise data centers.



Interview Questions on Load Balancing

1 Fundamentals

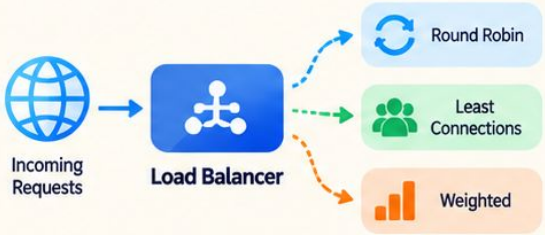


Q1 What is load balancing, and why is it important?

Q2 Explain the difference between Layer 4 and Layer 7 load balancing.

Q3 How does a load balancer handle high availability and failover?

2 Strategies & Use Cases



Q1 Compare Round Robin and Least Connections strategies.

Q2 What are the advantages of Weighted Load Balancing?

Q3 When would you use a software load balancer over a hardware one?

3 Real-World Scenarios



Q1 How would you design a scalable load balancing solution for a large e-commerce site?

Q2 What factors should be considered when choosing a load balancing strategy?

Q3 How does a load balancer improve security?

Key Takeaway | Think beyond the definition: explain trade-offs, failure handling, and real-world design choices.

Instructor: Rahul Rajat Singh

Summary & Key Takeaways

1. Load balancing enhances **scalability, reliability, and performance.**
2. Different types exist based on **layers and deployment models.**
3. Choosing the right strategy depends on **traffic patterns and system needs.**
4. Essential for **highly available and resilient** architectures.



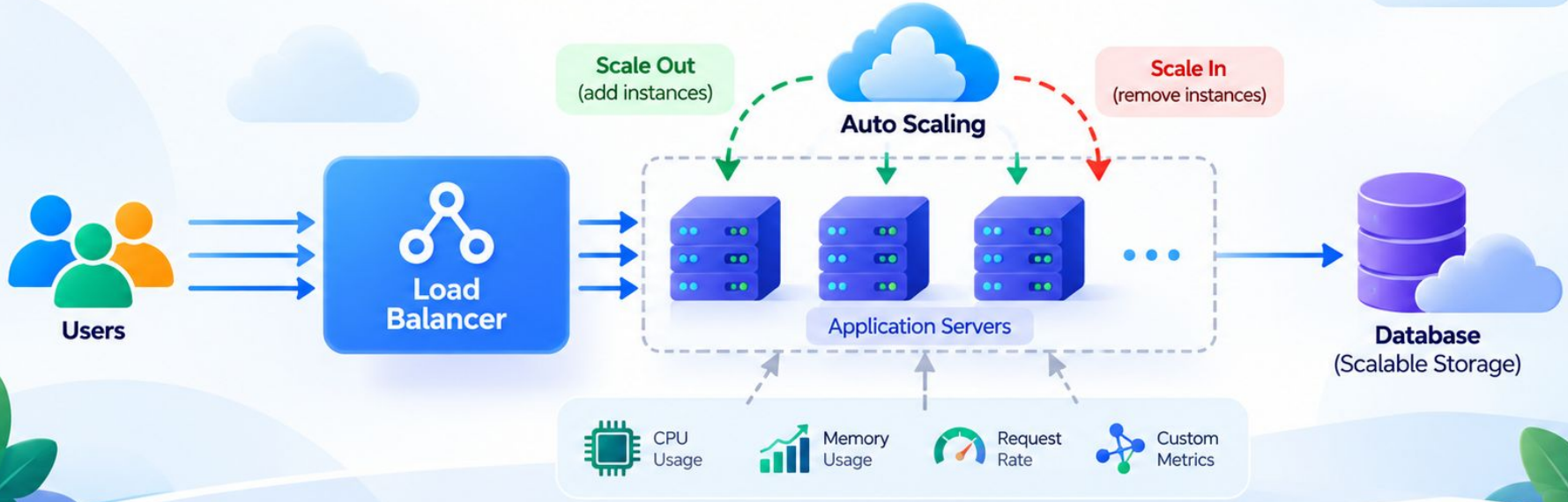
What's Next?

Autoscaling & Best Practices in Cloud Environments

Autoscaling & Best Practices in Cloud Environments

Scalability in System Design

Mastering System Design: From Basics to Cracking Interviews



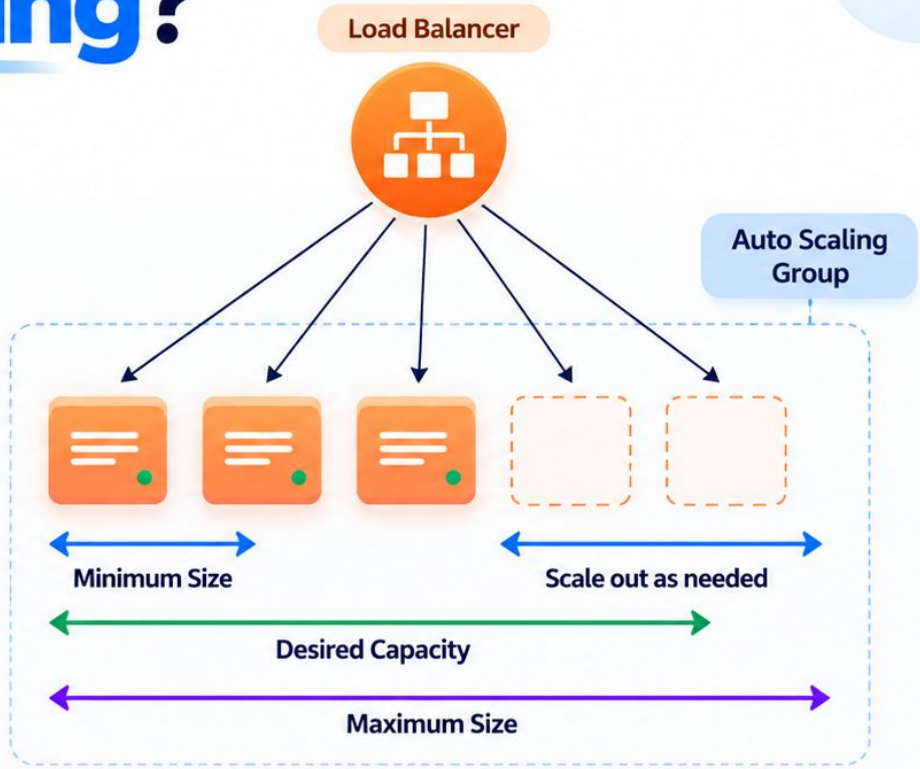
Instructor: Rahul Rajat Singh

What is Autoscaling?

Automatic adjustment of compute resources based on load.

Ensures **performance, availability,** and **cost-efficiency**.

Common in **microservices, web apps,** and event-driven systems.



How Autoscaling Works

1 Triggers

Autoscaling is triggered based on metrics that indicate higher or lower load.

- CPU usage
- Memory usage
- Request rate
- Queue length

2 Types

Autoscaling can scale out (add/remove instances) or scale up (resize a single instance).

Horizontal Scaling

Add or remove instances to handle load.

Vertical Scaling

Resize a single instance (more CPU/RAM).

3 Scaling Policies

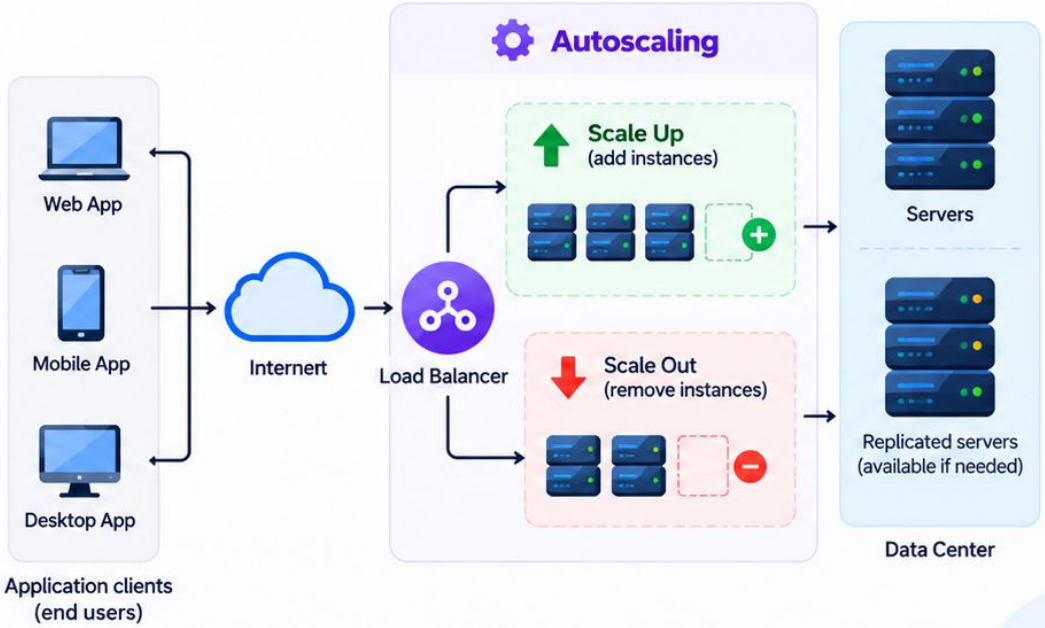
Autoscaling policies define when and how to scale.

Reactive

Scale based on thresholds (e.g., CPU > 70%).

Predictive

Scale based on trends and ML predictions.



Autoscaling Across Cloud Providers

Major cloud providers offer **built-in autoscaling** across compute and container services.

AWS

Auto Scaling for EC2, Lambda, ECS, EKS



- EC2 Auto Scaling
- AWS Lambda (automatic scaling)
- ECS Service Auto Scaling
- EKS Cluster Auto Scaling

Azure

Autoscaling via VM Scale Sets, App Services, AKS



- VM Scale Sets (Automatic Scaling)
- App Services (Auto Scale)
- AKS Cluster Auto Scaling

GCP

Autoscaling with MIGs, Cloud Run, GKE, Functions



- Managed Instance Groups (MIGs)
- Cloud Run (Automatic Scaling)
- GKE Cluster Autoscaler
- Cloud Functions (Automatic Scaling)

Key Takeaway | Scale capacity automatically as demand changes.

Instructor: Rahul Rajat Singh

Monitoring & Proactive Scaling

1 Use Metrics Like

CPU
Track CPU utilization

Memory
Monitor memory usage

Network
Track network in/out traffic

Queue Depth
Monitor message queue length

Custom KPIs
Business-specific metrics

2 Proactive Scaling

Predictive Algorithms
Scale based on ML models or usage trends

Scheduled Scaling
Scale based on known traffic patterns

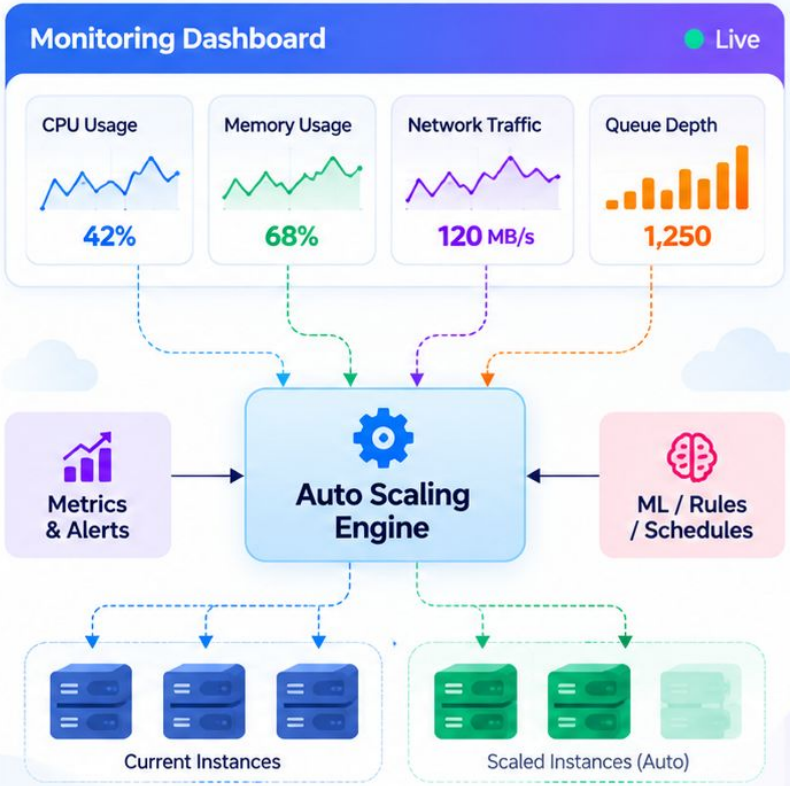
3 Tools for Monitoring & Scaling

CloudWatch
Monitoring & alarms (AWS)

Prometheus + Grafana
Metrics & dashboards

Azure Monitor
Monitoring & insights (Azure)

GCP Operations
Monitoring, logging & alerts (GCP)



Cost Optimization Strategies



1 Avoid Over-Provisioning

Scale just enough based on actual demand.



Right Sized (Optimal Cost)

Over-Provisioned (Higher Cost)

2 Use Spot/ Preemptible Instances

Ideal for batch workloads and non-critical tasks.



On-Demand $$$

Spot / Preemptible $

3 Apply Resource Limits & Quotas

Prevent unexpected costs with limits and budgets.



- CPU Limits
- Memory Quotas
- Budget Alerts

4 Rightsize Regularly

Review actual usage and remove/resize unused resources.

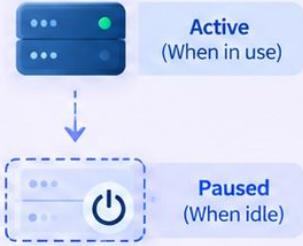


Before (Over-sized)

After (Right-sized)

5 Use Auto-Pausing or Scale-to-Zero

Automatically stop resources when idle.



Active (When in use)

Paused (When idle)

Key Takeaway | Optimize for the workload you have today, and continuously rightsize as usage patterns evolve.

Instructor: Rahul Rajat Singh

Interview Questions



1 Conceptual Understanding

- **1.1** What is autoscaling, and why is it important in distributed systems?
- **1.2** What's the difference between horizontal and vertical scaling?
- **1.3** How does predictive autoscaling work?

2 Cloud-Specific Scenarios

- **2.1** How does autoscaling work in AWS/Azure/GCP?
- **2.2** How would you set up autoscaling for a containerized application?
- **2.3** What metrics would you monitor for effective autoscaling?

3 Best Practices

- **3.1** How can you ensure cost optimization when implementing autoscaling?
- **3.2** What are some challenges with autoscaling in real-time systems?

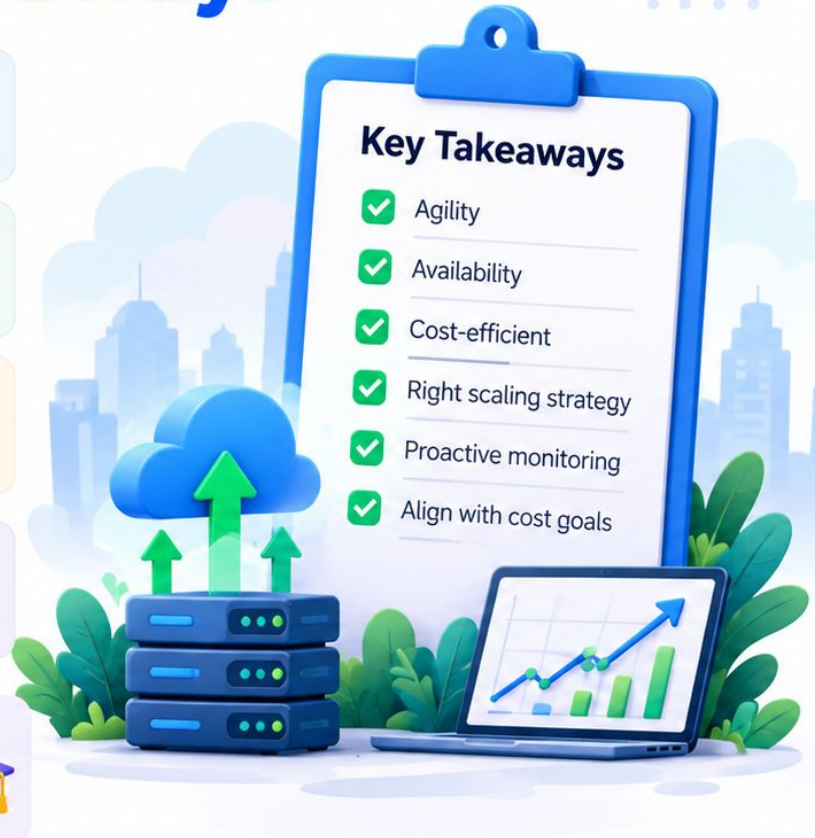
Summary and Key Takeaways

1. **Autoscaling ensures**
   agility, availability, and cost-efficiency.

2. **Choose the right scaling strategy**
   Horizontal, vertical, or predictive based on your needs.

3. **Monitor proactively**
   Stay ahead of load and scale before issues occur.

4. **Align with cost goals**
   Optimize autoscaling based on traffic patterns and budget.



What's Next?

Summary and recap - Scalability in system design.

Section Summary - Scalability in System Design

In this section, we explored the **core building blocks of scalability** in modern systems:

01 **Introduction to Scalability**
Why systems need to scale and the challenges that come with it.

02 **Scaling Strategies**
Deep dive into Horizontal, Vertical, and Diagonal scaling approaches.

03 **Load Balancing**
How different types of load balancers and algorithms distribute traffic efficiently.

04 **Autoscaling**
Best practices and how cloud platforms like AWS, Azure, and GCP support autoscaling.



What's Next?

Database and Storage